UNIVERSIDAD PERUANA LOS ANDES

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Trabajo de Base de datos

PRESENTADO POR:

Herrera bastidas fernando josue

CURSO:

Base de datos II

PRESENTADO A:

ING Raul Enrique Fernandez Bejarano

HUANCAYO – PERU

2026

INFORME TÉCNICO COMPARATIVO

Arquitecturas de Base de Datos:

Centralizada, Cliente-Servidor y Distribuida

Elaborado para:

Curso de Base de Datos

1. Introducción

Las bases de datos constituyen el núcleo de cualquier sistema de información moderno. La elección de la arquitectura de base de datos tiene un impacto directo en el rendimiento, la disponibilidad, la escalabilidad y el costo operativo de una solución tecnológica.

El presente informe técnico analiza y compara tres modelos arquitectónicos fundamentales: la arquitectura centralizada, la arquitectura cliente-servidor y la arquitectura distribuida. Para cada modelo se estudian sus características, ventajas, desventajas y ejemplos de implementación con DBMS actuales.

Objetivo: Determinar cuál arquitectura resulta más adecuada para un sistema de gestión académica, considerando los requerimientos de disponibilidad, rendimiento y mantenimiento.

2. Definición y Características de Cada Arquitectura

2.1 Arquitectura Centralizada

La arquitectura centralizada concentra todos los datos, la lógica de procesamiento y los recursos de almacenamiento en un único servidor o mainframe. Los usuarios acceden al sistema mediante terminales que no poseen capacidad de procesamiento independiente.

Características principales:

- Un único punto de control y administración de datos.
- Los clientes son terminales "tontas" sin procesamiento local.
- Toda la lógica de negocio reside en el servidor central.
- Alta consistencia de datos al existir una sola fuente de verdad.
- Fácil mantenimiento y actualización del sistema.

2.2 Arquitectura Cliente-Servidor

HUANCAYO – PERU

2026

En esta arquitectura, el procesamiento se divide entre el servidor (que gestiona los datos y la lógica de negocio) y los clientes (que gestionan la interfaz y parte de la lógica de presentación). La comunicación ocurre a través de una red mediante protocolos estándar como TCP/IP.

Características principales:

- Separación clara entre capa de presentación (cliente) y capa de datos (servidor).
- Los clientes poseen capacidad de procesamiento propia.
- Puede implementarse en dos capas (2-tier) o tres capas (3-tier).
- Escalabilidad horizontal mediante la incorporación de nuevos servidores.
- Soporte para múltiples usuarios concurrentes mediante gestión de conexiones.

2.3 Arquitectura Distribuida

La arquitectura distribuida dispersa los datos y el procesamiento entre múltiples nodos interconectados en red, que pueden estar en diferentes ubicaciones geográficas. Cada nodo puede funcionar de forma autónoma y participar en transacciones globales.

Características principales:

- Los datos se fragmentan (sharding) o replican entre nodos.
- No existe un punto central de fallo — alta tolerancia a fallos.
- Cumple parcialmente el Teorema CAP (Consistencia, Disponibilidad, Tolerancia a particiones).
- Escalabilidad prácticamente ilimitada mediante adición de nodos.
- Requiere mecanismos de sincronización y resolución de conflictos.

3. Ventajas y Desventajas en Distintos Entornos Tecnológicos

Centralizada	✔ Ventajas	✘ Desventajas
	• Administración simple y centralizada • Alta consistencia de datos • Bajo costo de implementación • Fácil respaldo y recuperación	• Punto único de fallo • Escalabilidad muy limitada • Cuello de botella con muchos usuarios • Baja disponibilidad ante fallos

Cliente-Servidor	✔ Ventajas	✘ Desventajas
	• Buena distribución de la carga • Flexibilidad para escalar • Seguridad centralizada en el servidor	• Servidor sigue siendo punto crítico • Mayor complejidad que arquitectura centralizada

	• Compatible con estándares SQL	• Latencia en redes de área amplia (WAN) • Licencias de software costosas

Distribuida	✔ Ventajas	✘ Desventajas
	• Alta disponibilidad y tolerancia a fallos • Escalabilidad horizontal casi ilimitada • Rendimiento óptimo con particionado • Sin punto único de fallo	• Alta complejidad de implementación • Consistencia eventual (no inmediata) • Costo de infraestructura elevado • Dificultad para garantizar transacciones ACID

4. Ejemplos de Implementación con DBMS Actuales

4.1 DBMS para Arquitectura Centralizada

Los sistemas de gestión de bases de datos orientados a arquitecturas centralizadas predominan en entornos de pequeña escala o en sistemas heredados (legacy):

- **SQLite:** Motor embebido de código abierto, sin servidor independiente. Ideal para aplicaciones móviles y de escritorio.
- **Microsoft Access:** DBMS de escritorio orientado a pequeñas organizaciones. Almacenamiento local en un único archivo .accdb.
- **dBASE / FoxPro:** Sistemas heredados que centralizaban el acceso desde múltiples terminales en redes locales.

4.2 DBMS para Arquitectura Cliente-Servidor

Esta es la categoría más amplia y adoptada en el mercado empresarial actual:

- **MySQL / MariaDB:** RDBMS open source ampliamente utilizado en aplicaciones web (stack LAMP/WAMP).
- **PostgreSQL:** Sistema objeto-relacional de código abierto con soporte para JSON, índices avanzados y extensiones.
- **Microsoft SQL Server:** Solución empresarial con integración nativa con el ecosistema Microsoft.
- **Oracle Database:** DBMS de nivel empresarial con capacidades avanzadas de particionado y alta disponibilidad (RAC).

4.3 DBMS para Arquitectura Distribuida

Los sistemas NoSQL y NewSQL han impulsado la adopción de arquitecturas distribuidas en entornos de alta demanda:

- **Apache Cassandra:** Base de datos NoSQL de columna ancha, diseñada para alta disponibilidad sin punto de fallo.
- **Google Spanner:** DBMS distribuido globalmente que garantiza consistencia fuerte con transacciones ACID.
- **CockroachDB:** Base de datos relacional distribuida compatible con PostgreSQL, con replicación automática.
- **MongoDB:** Base de datos documental distribuida con soporte para sharding y replicación integrados.

5. Tabla Comparativa General

La siguiente tabla resume los criterios más relevantes para la selección de arquitectura:

Criterio	Centralizada	Cliente-Servidor	Distribuida
Ubicación de datos	Un solo servidor central	Servidor(es) + clientes remotos	Múltiples nodos geográficos
Escalabilidad	Baja (vertical)	Media (horizontal limitada)	Alta (horizontal)
Disponibilidad	Baja — punto único de fallo	Media — depende del servidor	Alta — tolerancia a fallos
Rendimiento	Alto en redes locales	Bueno con carga balanceada	Muy alto con particionado
Consistencia de datos	Alta — fuente única de verdad	Alta en entornos controlados	Variable (CAP theorem)
Complejidad admin.	Baja	Media	Alta
Costo de impl.	Bajo	Medio	Alto
Ejemplo DBMS	SQLite, MS Access	MySQL, PostgreSQL, Oracle, SQL Srv	Cassandra, Google Spanner, CockroachDB
Caso de uso típico	Aplicaciones pequeñas / locales	Sistemas empresariales medianos	Sistemas globales / alta demanda

6. Recomendación para un Sistema de Gestión Académica

Un sistema de gestión académica (SGA) requiere administrar información de estudiantes, docentes, matrículas, calificaciones, horarios y recursos institucionales. A continuación se evalúa cuál arquitectura resulta más adecuada:

6.1 Requerimientos del sistema

HUANCAYO – PERU

2026

- Acceso concurrente de cientos a miles de usuarios (estudiantes, docentes, administrativos).
- Alta disponibilidad durante períodos críticos (matrículas, cierre de notas).
- Consistencia de datos en operaciones críticas (registros académicos, pagos).
- Posibilidad de crecimiento según el número de estudiantes y sedes.
- Costo de implementación y mantenimiento razonable para una institución educativa.

6.2 Arquitectura recomendada: Cliente-Servidor (3 capas)

Justificación: La arquitectura cliente-servidor de tres capas representa el equilibrio óptimo entre funcionalidad, costo y complejidad para un sistema de gestión académica de mediana escala.

Las razones que fundamentan esta elección son:

- Ofrece la consistencia de datos necesaria para registros académicos críticos, sin la complejidad operativa de una arquitectura distribuida.
- Permite escalar el servidor de datos de forma vertical (más recursos) o incorporar réplicas de lectura para períodos de alta demanda.
- Los DBMS como PostgreSQL o SQL Server, ampliamente documentados, reducen la curva de aprendizaje del equipo técnico.
- El costo de licenciamiento e infraestructura es significativamente menor al de una solución distribuida global.
- Permite implementar alta disponibilidad mediante clustering o replicación maestro-esclavo.

6.3 Consideración futura

Si la institución crece hacia múltiples sedes geográficas o supera los 50,000 usuarios concurrentes, se recomienda migrar gradualmente hacia una arquitectura distribuida con un DBMS como CockroachDB o Google Cloud Spanner, que mantiene compatibilidad SQL.

7. Conclusiones

- La arquitectura centralizada, aunque simple, resulta insuficiente para entornos con múltiples usuarios concurrentes o requerimientos de disponibilidad media-alta.
- La arquitectura cliente-servidor es la más adoptada en sistemas empresariales y educativos por su equilibrio entre rendimiento, consistencia y costo.
- La arquitectura distribuida ofrece la mayor escalabilidad y disponibilidad, pero a un costo de implementación y complejidad significativamente mayor.
- Para un sistema de gestión académica de mediana escala, la arquitectura cliente-servidor de tres capas con un DBMS relacional (PostgreSQL o SQL Server) constituye la solución más adecuada.
- La elección de arquitectura no es permanente: los sistemas deben diseñarse con una hoja de ruta que permita migrar hacia modelos distribuidos conforme crezca la demanda.

8. Referencias

HUANCAYO – PERU

2026

- Ramakrishnan, R. & Gehrke, J. (2003). Database Management Systems. McGraw-Hill.
- Silberschatz, A., Korth, H. & Sudarshan, S. (2019). Database System Concepts (7.ª ed.). McGraw-Hill.
- Google Cloud. (2024). Cloud Spanner: Distributed Relational Database. https://cloud.google.com/spanner
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. https://www.postgresql.org/docs/
- Apache Software Foundation. (2024). Apache Cassandra Documentation. https://cassandra.apache.org/doc/

HUANCAYO – PERU

2026